

DATA301-DATA471: Data Science in Practice

Assignment 4

Leon Cai (300692424)

Victoria University of Wellington
September 2026

1 Exercise 1

1.1 Order statistics for the quartiles [10%]

Summary of the two articles. Nine common definitions of the sample quantile are compared in Wicklin (2017), and they all follow the same general form given in Hyndman and Fan (1996) (H&F): $\hat{Q}(p) = (1 - \gamma)x_{(j)} + \gamma x_{(j+1)}$, where $x_{(j)}$ and $x_{(j+1)}$ are the order statistics closest to p and γ is an interpolation weight, with j and γ both depending on p , the sample size n , and a parameter m that is different for each definition ($j = \lfloor np + m \rfloor$).

The nine definitions split into two groups. Types 1-3 are discontinuous, meaning they jump between values instead of changing smoothly, since they come directly from the empirical CDF, and they always give a value that already appears in the data (or the average of two adjacent values) – this group is plotted as a staircase in Wicklin (2017). Types 4-9 are continuous and interpolate between order statistics using a *plotting position* p_k , the probability assigned to order statistic $x_{(k)}$, and these six types split further by how p_k is derived: types 6, 7 and 8 use a *distribution-free* rule (p_k does not depend on the population distribution F), while type 9 is built to be almost unbiased only for QQ-plots against a Normal distribution, so it depends on F instead.

Each definition is also checked in Hyndman and Fan (1996) against six properties a good sample quantile should have (continuity in p , symmetry in the tails, uniqueness, and matching the usual sample median at $p = 0.5$), and only type 5 satisfies all six, but it's a compromise that was not built from an estimation argument. Types 6-9 each satisfy five of the six and have a clearer theoretical basis, and of these, H&F recommend type 8 as a general default because its estimate of $Q(p)$ is approximately median-unbiased for *any* F – types 6 and 7 don't have this unbiased property, and type 9 is unbiased only for the Normal case. There's also a practical point in Wicklin (2017) that matters here: the nine definitions agree closely once n is fairly large (the article uses $n = 100$), but differ more for small samples or when the data have large gaps between neighbouring values, which includes the $n = 5$ cells used in the Monte Carlo comparison below.

Choice used in this report. Type 8 is used throughout (the default for `lambda_iqr()` above, and everywhere it is called), for two extra reasons on top of H&F's own recommendation: the Exponential distribution used here is clearly skewed, unlike the Normal that type 9 targets, and the smallest sample size in the design, $n = 5$, is exactly where Wicklin (2017) shows the choice of definition matters most. And this choice isn't only about this section – $\hat{\lambda}_{IQR}$ gets used again in the bootstrap correction below, and in every replication of the Monte Carlo experiment further on, so a bad quantile definition here would carry through to those results too.

Software default. R's `quantile()` defaults to type 7, confirmed below. This comes from

S-Plus, whose `quantile()` used type 7 (Hyndman and Fan (1996), Section 4), and R inherited it. Type 7 is not the type 8 that H&F recommend.

```
quantile(1:10) # uses R's default type

 0%   25%   50%   75%  100%
1.00  3.25  5.50  7.75 10.00

identical(quantile(1:10), quantile(1:10, type = 7)) # confirms default = type 7

[1] TRUE
```

1.2 Bootstrap bias-corrected estimator [20%]

Derivation. The bias of $\hat{\lambda}_{IQR}$ would ideally be found by simulating many samples from the true $\text{Exponential}(\lambda)$ distribution and comparing the average of $\hat{\lambda}_{IQR}$ across those samples to the true λ , as covered in the Week 9 lecture, but in practice there is only one sample $\mathbf{X}$ and the true λ is unknown. Bootstrap gets around this by treating $\mathbf{X}$ itself as a stand-in for the population, and $\hat{\lambda}_{IQR}(\mathbf{X})$ as a stand-in for the unknown true value. For a fixed sample $\mathbf{X}$ of size n , draw B bootstrap resamples $\mathbf{X}_1^*, \dots, \mathbf{X}_B^*$ (sampling n values from $\mathbf{X}$ with replacement). The bootstrap estimate of the bias of $\hat{\lambda}_{IQR}$ is

$$\widehat{\text{bias}} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*) - \hat{\lambda}_{IQR}(\mathbf{X}).$$

The first-order bias-corrected estimator subtracts this estimated bias:

$$\tilde{\lambda}(\mathbf{X}) = \hat{\lambda}_{IQR}(\mathbf{X}) - \widehat{\text{bias}} = 2\hat{\lambda}_{IQR}(\mathbf{X}) - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*).$$

```
lambda_ml <- function(x) 1 / mean(x)

lambda_iqr <- function(x, type = 8) {
  q <- quantile(x, probs = c(0.25, 0.75), type = type)
  log(3) / (q[[2]] - q[[1]])
}

lambda_tilde <- function(x, B = 200, type = 8) {
  n <- length(x)
  boot_est <- replicate(B, lambda_iqr(sample(x, n, replace = TRUE),
                                         type = type))
  # Small n: a bootstrap resample can tie at the quartiles (q3=q1), making
  # lambda_iqr(.) = Inf. Drop these before averaging so one Inf doesn't
  # dominate the correction.
  boot_est <- boot_est[is.finite(boot_est)]
  2 * lambda_iqr(x, type = type) - mean(boot_est)
}

x <- rexp(30, rate = 2)
c(ML = lambda_ml(x), IQR = lambda_iqr(x), bias_corrected = lambda_tilde(x))
```

ML	IQR	bias_corrected
1.562896	1.741311	1.814164

1.3 Monte Carlo comparison [20%]

Design. The structure follows the course's own Monte Carlo tutorial (Frery 2026): for each situation $(\lambda, n) \in \Lambda \times N$, R independent replications are generated, each replication gives one sample $\mathbf{X}$ of size n and one value of each of the three estimators, and the MSE for that situation is the sample mean of $(\hat{\lambda} - \lambda)^2$ over the R replications.

$R = 5000$ is used for every situation here, straight from the Week 9 lecture, where the lecturer said 5000 replications per case is enough for an assignment this size, and that you only need to control R more tightly in much bigger experiments. It also leaves a decent margin on precision: the standard error of the Monte Carlo estimate of MSE is about $\sqrt{\widehat{\text{Var}}[(\hat{\lambda} - \lambda)^2]/R}$, and even at $n = 5$, where the estimators are most variable, $R = 5000$ keeps this small next to the MSE values being compared.

```
mc_mse <- function(lambda, n, R, B = 200, type = 8) {
  est <- matrix(NA_real_, nrow = R, ncol = 3,
               dimnames = list(NULL, c("ML", "IQR", "bias_corrected")))
  for (r in seq_len(R)) {
    x <- rexp(n, rate = lambda)
    est[r, ] <- c(lambda_ml(x), lambda_iqr(x, type = type),
                  lambda_tilde(x, B = B, type = type))
  }
  colMeans((est - lambda)^2)
}

lambda_grid <- c(0.1, 1, 10)
n_grid <- c(5, 30, 100)
R <- 5000

results <- do.call(rbind, lapply(lambda_grid, function(lam) {
  do.call(rbind, lapply(n_grid, function(n) {
    mse <- mc_mse(lam, n, R = R)
    data.frame(lambda = lam, n = n, R = R, t(mse), check.names = FALSE)
  }))
}))
results
```

	lambda	n	R		ML	IQR	bias_corrected
1	0.1	5	5000	6.350623e-03	1.256162e-02	1.869204e+01	
2	0.1	30	5000	3.995203e-04	1.002016e-03	1.242778e-03	
3	0.1	100	5000	1.080374e-04	2.360035e-04	2.788539e-04	
4	1.0	5	5000	5.684110e-01	9.374353e-01	4.749525e+03	
5	1.0	30	5000	3.783969e-02	9.543773e-02	1.185430e-01	
6	1.0	100	5000	1.065686e-02	2.300534e-02	2.690247e-02	
7	10.0	5	5000	6.177431e+01	9.638678e+01	7.003303e+04	
8	10.0	30	5000	4.012428e+00	9.753578e+00	1.229160e+01	
9	10.0	100	5000	1.081235e+00	2.446096e+00	2.822481e+00	

The same numbers are easier to compare as a plot, especially since `bias_corrected`'s MSE spans several orders of magnitude across the design – both axes below are on a log scale.

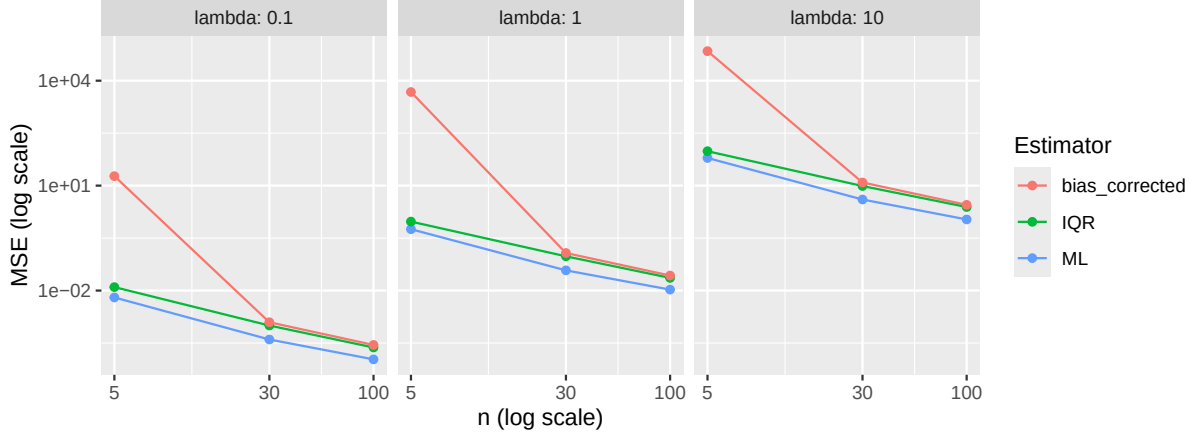
```
est_names <- c("ML", "IQR", "bias_corrected")
results_long <- do.call(rbind, lapply(est_names, function(est) {
  data.frame(lambda = results$lambda, n = results$n,
```

```

    estimator = est, mse = results[[est]])
}))

ggplot(results_long, aes(n, mse, color = estimator)) +
  geom_line() + geom_point() +
  scale_x_log10(breaks = c(5, 30, 100)) +
  scale_y_log10() +
  facet_wrap(~ lambda, labeller = label_both) +
  labs(x = "n (log scale)", y = "MSE (log scale)", color = "Estimator")

```



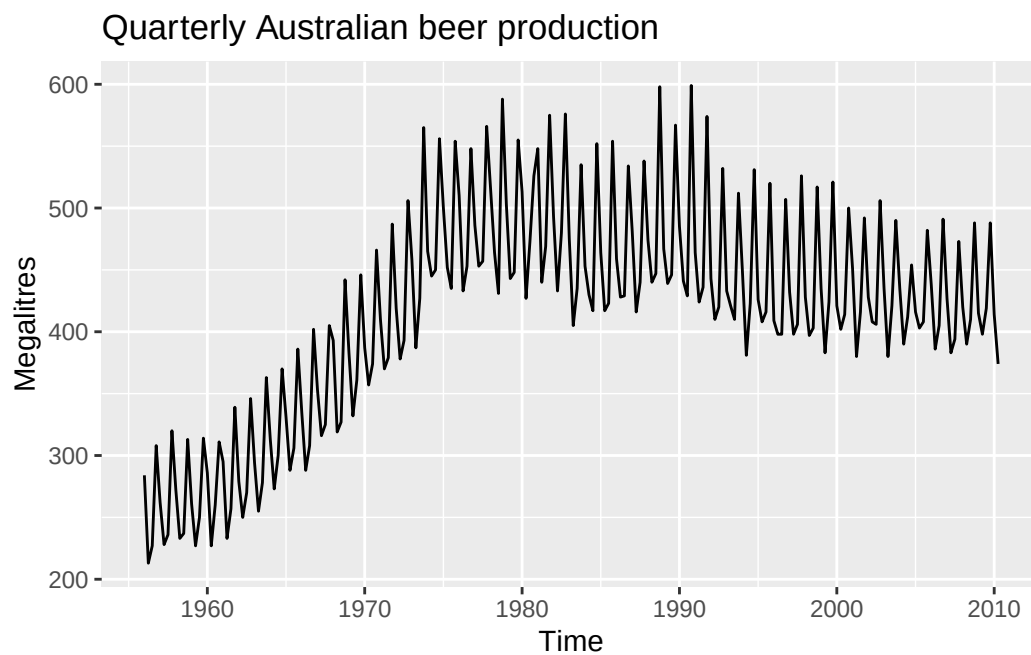
Discussion. $\hat{\lambda}_{ML}$ has the lowest MSE in every one of the nine cells. This matches theory. It is the maximum likelihood estimator, so it uses the full sample through the likelihood, and it has the smallest asymptotic variance among regular estimators. $\hat{\lambda}_{IQR}$ only uses two order statistics, so it should do worse. The MSE gap between the two stays in a similar range across the design: $\hat{\lambda}_{IQR}$'s MSE is roughly 1.6 to 2.5 times $\hat{\lambda}_{ML}$'s, at every (λ, n) combination, with no clear trend as n or λ change. This roughly matches the idea that the IQR estimator has a fixed *asymptotic* efficiency loss from throwing away most of the sample, but that result only has to hold as $n \rightarrow \infty$ – some of the spread in the ratio seen here could be genuine finite-sample behaviour rather than just Monte Carlo noise from a design that is too small to pin the ratio down exactly.

The bias-corrected estimator $\tilde{\lambda}$ is *worse* than the uncorrected $\hat{\lambda}_{IQR}$ at every single (λ, n) in the design, not only at $n = 5$. At $n = 100$, the correction makes MSE about 15-18% bigger than $\hat{\lambda}_{IQR}$, across all three λ values. At $n = 30$, the gap is about 24-26%. At $n = 5$, it is much worse: three to four orders of magnitude bigger. The reason is a bias-variance trade-off that goes the wrong way. $\hat{\lambda}_{IQR}$ is a nonlinear function (a reciprocal) of the sample's interquartile spread. A bootstrap resample can, just by chance, have a noticeably smaller or larger spread than the original sample. After taking the reciprocal, and then doubling it in $\tilde{\lambda} = 2\hat{\lambda}_{IQR} - \overline{\hat{\lambda}_{IQR}^*}$, this small chance difference turns into a large swing in the correction term. The variance this adds is bigger than the bias it removes, so MSE gets worse overall, and more so as n shrinks, because bootstrap resamples are then more likely to have an unusually small spread. So the first-order bootstrap correction does not help this estimator, at any of the sample sizes tested here.

All three estimators' MSE decreases roughly with $1/n$, which matches the usual parametric convergence rate, and it also increases roughly with λ^2 : at $n = 5$, $\text{MSE}(\hat{\lambda}_{ML})/\lambda^2$ is about 0.64, 0.57, and 0.62 at $\lambda = 0.1, 1, 10$, and these are close enough that the ratio looks roughly constant. That makes sense since $\hat{\lambda}$ estimates a rate (units of $1/\text{time}$), so squared error should scale with λ^2 under a simple change of the time unit.

2 Exercise 2

```
autoplot(ausbeer) + labs(title = "Quarterly Australian beer production", y = "Megalitres")
```



2.1 Frequency

TODO

2.2 Trend

TODO

2.3 Seasonality

TODO

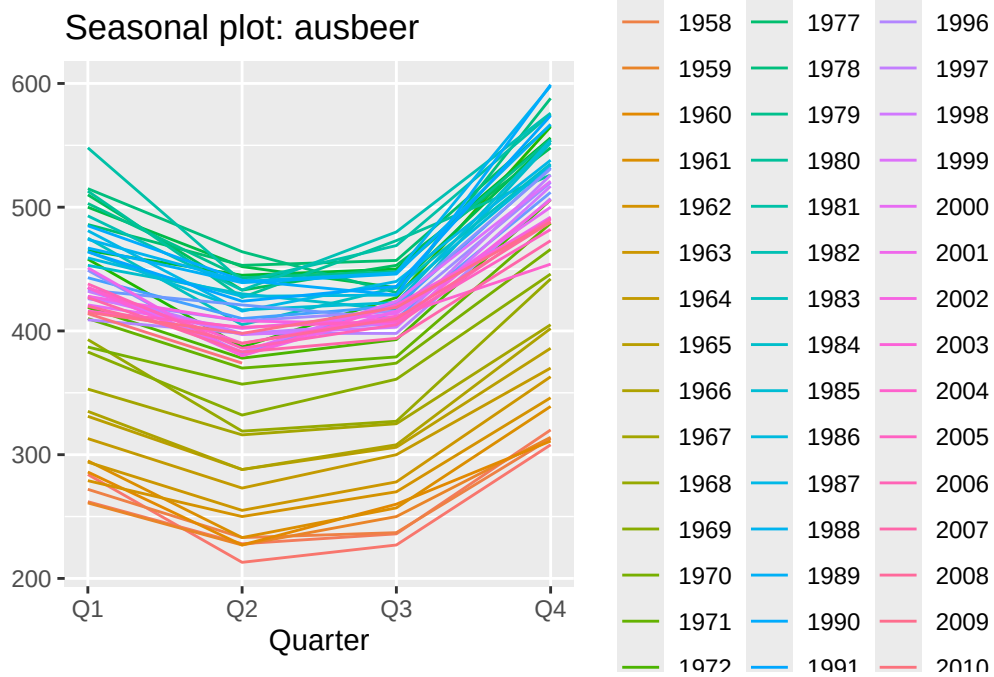
2.4 Variance stability

TODO

2.5 Unusual observations

TODO

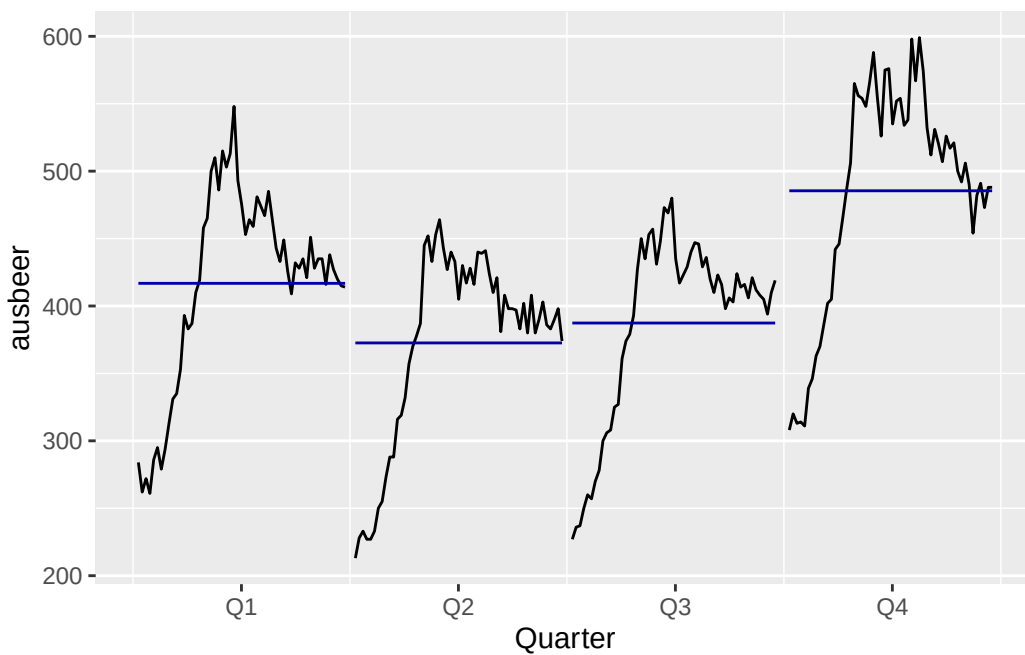
```
ggseasonplot(ausbeer)
```



2.6 Seasonal plot

TODO

```
ggsubseriesplot(ausbeer)
```



2.7 Seasonal subseries plot

TODO

2.8 Detrend / transform?

TODO

References

- Frery, Alejandro C. 2026. *The Monte Carlo Method*. DATA301/DATA471 tutorial slides. Victoria University of Wellington.
- Hyndman, Rob J., and Yanan Fan. 1996. “Sample Quantiles in Statistical Packages.” *The American Statistician* 50 (4): 361–65. <https://doi.org/10.2307/2684934>.
- Wicklin, Rick. 2017. *Sample Quantiles: A Comparison of 9 Definitions*. SAS Blogs, The DO Loop. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>.